

```

"""
validaciones.py
=====

Validaciones de INTEGRIDAD del panel, previas a cualquier calculo de metricas.

Se comprueba, en este orden:

1. Que existan las columnas declaradas como target, id y tiempo.
2. Que el target sea utilizable (no constante, no 100% nulo).
3. Que la llave compuesta ``id + tiempo`` no tenga duplicados indebidos.
4. Que la estructura de panel sea coherente (entidades, periodos, balance).

El resultado no es solo "pasa / no pasa": se devuelve un reporte con hallazgos
que se vuelca a la bitacora, porque un panel desbalanceado o con huecos no
invalida el analisis pero SI condiciona como se interpretan las metricas.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger

LOGGER = obtener_logger("validaciones")

class ErrorValidacionPanel(ValueError):
    """La estructura del panel impide continuar con el proceso."""

@dataclass
class ReporteValidacion:
    """Hallazgos de la validacion estructural del panel."""

    hallazgos: list[dict[str, Any]] = field(default_factory=list)
    duplicados_llave: int = 0
    n_entidades: int = 0
    n_periodos: int = 0
    panel_balanceado: bool = False
    tipo_target: str = ""

    def agregar(self, chequeo: str, resultado: str, detalle: str, severidad: str = "INFO") -> None:
        """Registra un hallazgo. `severidad` en {INFO, ADVERTENCIA, ERROR}."""
        self.hallazgos.append(
            {
                "chequeo": chequeo,
                "resultado": resultado,
                "detalle": detalle,
                "severidad": severidad,
            }
        )
        nivel = {"INFO": LOGGER.info, "ADVERTENCIA": LOGGER.warning, "ERROR": LOGGER.error}[severidad]
        nivel("[validacion] %s -> %s. %s", chequeo, resultado, detalle)

    def a_dataframe(self) -> pd.DataFrame:
        return pd.DataFrame(self.hallazgos)

# -----
# Validacion principal
# -----
def validar_panel(df: pd.DataFrame, cfg: ConfigPipeline) -> ReporteValidacion:
    """Ejecuta la bateria completa de validaciones estructurales del panel.

    Raises

```

```

-----
ErrorValidacionPanel
    Ante fallos que hacen imposible continuar (columnas ausentes, target
        degenerado, duplicados exactos en la llave del panel).
"""
rep = ReporteValidacion()

# -----
# 1. Existencia de las columnas de rol
# -----
requeridas = {
    "columna_target": cfg.columna_target,
    "columna_id": cfg.columna_id,
    "columna_tiempo": cfg.columna_tiempo,
}
faltantes = {k: v for k, v in requeridas.items() if v not in df.columns}
if faltantes:
    disponibles = ", ".join(map(str, df.columns[:40]))
    raise ErrorValidacionPanel(
        "Columnas declaradas que no existen en el dataset: "
        + "; ".join(f"{k}='{v}'" for k, v in faltantes.items())
        + f"\nColumnas disponibles (primeras 40): {disponibles}"
    )
rep.agregar(
    "Existencia de columnas de rol", "OK",
    f"target='{cfg.columna_target}', id='{cfg.columna_id}', tiempo='{cfg.columna_tiempo}'",
)

# Exclusiones manuales inexistentes: no es bloqueante, pero hay que avisar.
exc_inexistentes = [c for c in cfg.columnas_excluidas if c not in df.columns]
if exc_inexistentes:
    rep.agregar(
        "Columnas excluidas manualmente", "ADVERTENCIA",
        f"No existen en el dataset y se ignoran: {exc_inexistentes}", "ADVERTENCIA",
    )

# -----
# 2. Utilidad del target
# -----
y = df[cfg.columna_target]
pct_nulos_y = float(y.isna().mean())
if pct_nulos_y == 1.0:
    raise ErrorValidacionPanel(f"El target '{cfg.columna_target}' es 100% nulo.")
if y.dropna().nunique() <= 1:
    raise ErrorValidacionPanel(
        f"El target '{cfg.columna_target}' es constante (un solo valor). "
        "No hay nada que discriminar."
    )
if pct_nulos_y > 0:
    rep.agregar(
        "Nulos en el target", "ADVERTENCIA",
        f"{pct_nulos_y:.2%} de filas con target nulo; se excluyen de las fases bivariada, "
        "multivariada y Boruta (no del diagnostico).",
        "ADVERTENCIA",
    )
else:
    rep.agregar("Nulos en el target", "OK", "El target no tiene valores nulos.")

from .io_utils import detectar_tipo_target # import local: evita ciclo

rep.tipo_target = detectar_tipo_target(y)
detalle_target = f"tipo detectado='{rep.tipo_target}', valores unicos={y.dropna().nunique()}"
if rep.tipo_target == "BINARIO":
    tasa = float(pd.to_numeric(y, errors="coerce").dropna().mean())
    detalle_target += f", tasa de evento={tasa:.4%}"
    if min(tasa, 1 - tasa) < 0.01:
        rep.agregar(
            "Balance del target", "ADVERTENCIA",
            f"Clase minoritaria en {min(tasa, 1-tasa):.4%}. Con eventos tan escasos el IV y "
            "el Gini se vuelven inestables por bin; considere agrupar periodos.",
            "ADVERTENCIA",

```

```

    )
rep.agregar("Tipo de target", "OK", detalle_target)

# -----
# 3. Duplicados en la llave compuesta id + tiempo
# -----
llave = [cfg.columna_id, cfg.columna_tiempo]
dup_mask = df.duplicated(subset=llave, keep=False)
rep.duplicados_llave = int(dup_mask.sum())

if rep.duplicados_llave > 0:
    # Se distingue duplicado EXACTO (fila repetida entera) de duplicado de
    # llave con contenido distinto. El primero es basura de carga; el
    # segundo significa que la granularidad declarada del panel es erronea.
    dup_exactos = int(df[dup_mask].duplicated(keep=False).sum())
    ejemplos = (
        df.loc[dup_mask, llave].drop_duplicates().head(5).astype(str)
        .agg(" | ".join, axis=1).tolist()
    )
    rep.agregar(
        "Duplicados en llave id+tiempo", "ERROR",
        f"{rep.duplicados_llave} filas comparten la llave ({cfg.columna_id}, {cfg.columna_tiempo}); "
        f"de ellas {dup_exactos} son filas identicas. Ejemplos: {ejemplos}",
        "ERROR",
    )
    raise ErrorValidacionPanel(
        f"La llave del panel ({cfg.columna_id}, {cfg.columna_tiempo}) no es unica: "
        f"{rep.duplicados_llave} filas duplicadas. Un panel exige UNA observacion por "
        "entidad y periodo. Corrija el origen o agregue previamente."
    )
rep.agregar(
    "Duplicados en llave id+tiempo", "OK",
    f"La llave ({cfg.columna_id}, {cfg.columna_tiempo}) es unica en las {len(df)} filas.",
)

# -----
# 4. Coherencia de la estructura de panel
# -----
rep.n_entidades = int(df[cfg.columna_id].nunique())
rep.n_periodos = int(df[cfg.columna_tiempo].nunique())

if rep.n_periodos < 2:
    rep.agregar(
        "Dimension temporal", "ADVERTENCIA",
        f"Solo hay {rep.n_periodos} periodo(s). El dataset es transversal en la practica: "
        "las metricas de estabilidad temporal (PSI, IV por periodo) no seran informativas.",
        "ADVERTENCIA",
    )
else:
    rep.agregar(
        "Dimension temporal", "OK",
        f"{rep.n_periodos} periodos distintos en '{cfg.columna_tiempo}'.",
    )

if rep.n_entidades < 2:
    rep.agregar(
        "Dimension de entidad", "ADVERTENCIA",
        f"Solo hay {rep.n_entidades} entidad(es): serie de tiempo, no panel.", "ADVERTENCIA",
    )
else:
    rep.agregar(
        "Dimension de entidad", "OK",
        f"{rep.n_entidades} entidades distintas en '{cfg.columna_id}'.",
    )

# Balance: un panel es balanceado si toda entidad aparece en todo periodo.
esperado = rep.n_entidades * rep.n_periodos
rep.panel_balanceado = (esperado == len(df))
obs_por_entidad = df.groupby(cfg.columna_id, observed=True).size()
rep.agregar(
    "Balance del panel",

```

```

"BALANCEADO" if rep.panel_balanceado else "DESBALANCEADO",
(
    f"filas={len(df)}, esperado si fuera balanceado={esperado} "
    f"({rep.n_entidades} entidades x {rep.n_periodos} periodos); "
    f"observaciones por entidad: min={int(obs_por_entidad.min())}, "
    f"mediana={int(obs_por_entidad.median())}, max={int(obs_por_entidad.max())}. "
    + (
        "Panel completo."
        if rep.panel_balanceado
        else "Panel incompleto: las entidades con mas periodos pesan mas en las metricas "
            "agrupadas (pooled). Se documenta para la interpretacion."
    )
),
"INFO" if rep.panel_balanceado else "ADVERTENCIA",
)

# Nulos en las propias llaves: rompen la trazabilidad del panel.
for col in (cfg.columna_id, cfg.columna_tiempo):
    n_nulos = int(df[col].isna().sum())
    if n_nulos:
        rep.agregar(
            f"Nulos en '{col}':", "ADVERTENCIA",
            f"{n_nulos} filas sin valor de llave: no pueden asignarse a una entidad/periodo.",
            "ADVERTENCIA",
        )

# Columnas duplicadas por nombre: pandas las admite y despues rompen todo.
dup_cols = df.columns[df.columns.duplicated()].tolist()
if dup_cols:
    raise ErrorValidacionPanel(f"Hay nombres de columna repetidos en el dataset: {dup_cols}.")

rep.agregar("Nombres de columna unicos", "OK", f"{df.shape[1]} columnas sin nombres repetidos.")
return rep

def obtener_columnas_candidatas(
    df: pd.DataFrame, cfg: ConfigPipeline, tipos: dict[str, str]
) -> list[str]:
    """Devuelve las columnas evaluables como variables explicativas.

    Se excluyen: target, id, tiempo y las exclusiones manuales del usuario.
    Los nombres provienen SIEMPRE de la configuracion, nunca de literales.
    """
    reservadas = set(cfg.columnas_no_candidatas)
    candidatas = [c for c in df.columns if c not in reservadas]
    LOGGER.info(
        "Columnas candidatas: %d de %d (excluidas %d por rol/configuracion: %s).",
        len(candidatas), df.shape[1], len(reservadas & set(df.columns)),
        sorted(reservadas & set(df.columns)),
    )
    return candidatas

def calcular_varianza_panel(
    serie: pd.Series, ids: pd.Series
) -> tuple[float, float, float]:
    """Descompone la varianza de una variable en componentes de panel.

    En datos de panel la varianza total se descompone en:

    - **Between** (entre entidades): varianza de las medias por entidad.
      Capta diferencias estructurales y permanentes entre unidades.
    - **Within** (intra entidad): varianza promedio dentro de cada entidad.
      Capta la evolucion temporal de cada unidad.

    Es una distincion que un analisis transversal ignora, y es importante:
    una variable con varianza total alta pero varianza *within* nula es
    **invariante en el tiempo** (un atributo fijo de la entidad, como el
    sector economico). No se elimina por ello, pero condiciona el tipo de
    modelo que puede aprovecharla (p. ej. un modelo de efectos fijos la
    absorbe por completo).

```

Returns

```
(var_within, var_between, icc)
    ``icc`` = correlacion intraclass = var_between / (var_between + var_within).
    Cercano a 1 -> la variable es casi fija por entidad.
    Cercano a 0 -> casi toda la variacion es temporal.
    """
s = pd.to_numeric(serie, errors="coerce")
valido = s.notna()
if valido.sum() < 2:
    return np.nan, np.nan, np.nan

s, g = s[valido], ids[valido]
medias = s.groupby(g, observed=True).transform("mean")

var_between = float(np.nanvar(s.groupby(g, observed=True).mean(), ddof=0))
var_within = float(np.nanvar(s - medias, ddof=0))

denom = var_between + var_within
icc = float(var_between / denom) if denom > 0 else np.nan
return var_within, var_between, icc
```